

Trabalho Prático 1

Ítalo Dell’Areti

Universidade Federal de Minas Gerais (UFMG)

Belo Horizonte – MG – Brasil

1. Introdução

O problema proposto consiste em desenvolver um sistema de comunicação cliente-servidor para simular combates espaciais. O sistema deve implementar um protocolo de rede customizado (StarFleet Protocol) que permita a troca de mensagens entre jogadores e servidor, gerenciando batalhas por turnos entre naves espaciais. O protocolo deve resolver quatro aspectos principais:

- Estabelecer comunicação confiável entre cliente e servidor usando TCP/IP.
- Implementar suporte dual-stack (IPv4 e IPv6) para máxima compatibilidade.
- Gerenciar estado de combate com regras complexas envolvendo cinco ações possíveis (Laser, Torpedo, Shield, Cloak, Jump).
- Garantir sincronização correta entre cliente e servidor através de mensagens estruturadas.

Ao final, o sistema deve processar todas as interações de combate e retornar resultados precisos para os jogadores, incluindo inventário final e condições de término. Portanto, com base nas características de protocolos de rede, arquitetura cliente-servidor, sockets TCP e programação em C, implementamos uma solução que respeita todas as nuances do problema. Tendo em vista disso, este documento possui como objetivo explicitar o funcionamento desta implementação.

2. Modelagem

2.1 Arquitetura do Sistema

O sistema segue o modelo cliente-servidor tradicional com comunicação TCP/IP, suportando IPv4 e IPv6. A estrutura modular é composta por quatro componentes: diretório app com a camada de aplicação (client.c e server.c), diretório core com o núcleo do protocolo (protocol.c/h para mensagens e game.c/h para lógica de combate), diretório bin para executáveis gerados, e Makefile para automação do build.

A separação de responsabilidades garante que cada módulo tenha função específica, protocolo define mensagens e validações, game implementa regras através de tabela de decisão, servidor gerencia conexões e estado da partida, e cliente cuida da interface e conexão dual-stack.

2.2 Protocolo de Comunicação

O protocolo utiliza mensagens binárias estruturadas (**BattleMessage**) contendo tipo, ações, pontos de vida, contadores e mensagem textual. Define sete tipos de mensagens: **MSG_INIT** (boas-vindas), **MSG_ACTION_REQ** (solicita ação), **MSG_ACTION_RES** (resposta do cliente), **MSG_BATTLE_RESULT** (resultado do turno), **MSG_INVENTORY** (estatísticas finais), **MSG_ESCAPE** (fuga) e **MSG_GAME_OVER** (fim de jogo).

A estrutura **BattleMessage** contém oito campos principais:

- **type** (tipo da mensagem enum)
- **client_action** e **server_action** (inteiros de 0 a 4 representando as ações)
- **client_hp** e **server_hp** (pontos de vida atuais)
- **client_torpedoes** e **client_shields** (contadores de uso)
- **message** (array de 256 caracteres para mensagens descritivas).

O fluxo inicia com conexão do cliente, seguido de loop de turnos onde servidor solicita ação, cliente responde, servidor processa e retorna resultado. Ao final, o servidor envia inventário seguido de escape ou game over antes de encerrar a conexão.

2.3 Regras de Jogo

O combate é baseado em 100 HP iniciais, com dano padrão de 20 HP por ataque bem-sucedido. Cinco ações estão disponíveis:

- Laser Attack (penetra camuflagem)
- Photon Torpedo (vence laser mas é bloqueado por camuflagem)
- Shields Up (bloqueia todos ataques)
- Cloaking (evita torpedos mas vulnerável a laser)
- Hyper Jump (fuga imediata).

As interações seguem regras específicas: Laser vs Laser causa dano mútuo, Torpedo vence Laser, Laser penetra Cloaking, Torpedo é bloqueado por Shield e Cloaking, Shield bloqueia tudo, e Hyper Jump encerra partida independente da ação adversária.

2.4 Comportamento Dual-Stack IPv4/IPv6

O sistema implementa suporte nativo para IPv4 e IPv6, porém com comportamentos distintos dependendo da versão do protocolo escolhida. O servidor IPv4 (./server v4 porta) cria um socket exclusivamente IPv4 usando AF_INET, aceitando apenas conexões de clientes IPv4.

Já o servidor IPv6 (./server v6 porta) utiliza AF_INET6 e, por padrão, habilita o modo dual-stack. O modo dual-stack permite aceitar simultaneamente conexões IPv4 e IPv6. Quando um cliente IPv4 conecta a um servidor IPv6, o endereço IPv4 é

automaticamente mapeado para o formato IPv6 através do mecanismo de IPv4-mapped IPv6 addresses. A implementação não utiliza a flag `IPV6_V6ONLY`, que desabilitaria o dual-stack e forçaria o servidor IPv6 a aceitar apenas conexões IPv6. Esta decisão mantém o comportamento padrão do sistema operacional, maximizando a compatibilidade sem adicionar complexidade ao código.

3. Implementação

3.1 Principais Decisões e Escolhas

A implementação do protocolo StarFleet baseou-se em escolhas fundamentais que definiram a arquitetura e qualidade da solução. A decisão mais importante foi adotar abordagem table-driven para a lógica de combate, centralizando todas as regras em estrutura de dados declarativa ao invés de código imperativo com condicionais.

Esta escolha transformou várias linhas de if-else encadeados em simples tabela de consulta, onde cada linha representa uma regra completa com ações, danos, flags de término e mensagens descritivas. A legibilidade foi melhorada e adicionar ou modificar regras tornou-se questão de editar linhas da tabela sem alterar lógica de código.

Outra decisão fundamental foi o uso de wildcards (valor -1) na tabela de regras para evitar repetição massiva. Situações como fuga versus qualquer ação não-fuga, que exigiram 4 linhas repetitivas, foram reduzidas a uma única linha genérica. O algoritmo de busca foi projetado para primeiro tentar match exato entre as ações e, caso não encontre, buscar matches usando wildcards.

A escolha pela centralização completa de estado no servidor foi importante para simplificar a arquitetura. Toda informação da partida (HP, contadores, flags, turnos) é mantida apenas no servidor através da estrutura **BattleSession**, sendo enviada completa em cada mensagem.

O cliente foi projetado como componente stateless que apenas exibe informações recebidas e envia ações. Esta decisão eliminou problemas de sincronização e inconsistências entre cliente e servidor.

O suporte IPv4/IPv6 foi implementado desde o início usando o **getaddrinfo** com flag **AF_UNSPEC**. Esta escolha garantiu que o cliente tente automaticamente todas as opções de endereçamento disponíveis (IPv4 e IPv6), enquanto o servidor aceita configuração por argumento de linha de comando.

A padronização de nomenclatura das variáveis e funções evita nomes não descritivos e ambiguidades, tornando o código auto documentado. Dessa forma, tendo impacto direto na legibilidade, permitindo que qualquer desenvolvedor compreenda o propósito de cada variável ou função.

O servidor foi implementado como single-threaded processando uma conexão por vez por simplicidade e adequação ao escopo do trabalho, onde múltiplas conexões simultâneas não eram requisito.

3.2 Desafios e Soluções Implementadas

O principal desafio foi garantir a consistência da estrutura **BattleMessage** entre cliente e servidor compilados separadamente. A solução implementada evitou tipos complexos, usando exclusivamente inteiros e arrays de caracteres que possuem layout de memória previsível.

O servidor gerencia uma conexão por vez através de socket configurado com **BACKLOG=1**, aguardando conexão, processando partida completa e encerrando antes de aceitar próximo cliente.

A estrutura **BattleSession** armazena todo estado da partida incluindo **client_health** e **server_health** (HP atual de ambos), **client_torpedo_count** e **client_shield_count** (contadores de uso), **round_num** (turno atual), e **client_fled** e **server_fled** (flags de fuga). Esta centralização eliminou a necessidade de sincronização complexa entre múltiplos clientes.

Ambos os componentes foram compilados com flags idênticas garantindo o mesmo comportamento de padding e alinhamento. O gerenciamento de todas as combinações de regras de combate com comportamentos assimétricos e especiais exigiu abordagem sistemática. A solução foi criar uma matriz completa das interações entre ações, implementar cada regra explicitamente como linha da tabela.

A separação correta de mensagens de término conforme especificação do protocolo foi resolvida implementando envio sequencial de duas mensagens ao final da partida. Primeiro o servidor envia **MSG_INVENTORY** contendo estatísticas completas (HP restante, torpedos e escudos usados, número de turnos), seguido de **MSG_ESCAPE** para situações de fuga ou **MSG_GAME_OVER** para término por destruição. Esta separação garante que o cliente sempre receba inventário independente do tipo de término.

A mitigação de condição de corrida foi alcançada através de design de protocolo estritamente síncrono request-response. O cliente sempre aguarda resposta completa do servidor antes de solicitar nova ação do usuário, e o servidor processa uma conexão por vez. Esta escolha facilitou a implementação eliminando a necessidade de sincronização complexa.

3.3 Estratégias de Organização

A estrutura foi organizada separando claramente responsabilidades em camadas lógicas. O diretório core contém lógica de negócio que permite a reutilização por diferentes tipos de cliente ou servidor. O diretório app contém implementações específicas de cliente e servidor que utilizam os módulos core.

Esta ideia permitiu validar cada camada antes de adicionar a próxima, reduzindo a complexidade de debugging. A fase final dedicada exclusivamente a refatoração e limpeza garantiu que código morto fosse removido e nomenclatura ficasse consistente. A escolha por funções static inline para operações pequenas e frequentes trouxe benefícios duplos de performance e encapsulamento

4. Possíveis Melhorias

Uma melhoria importante seria tornar o servidor mais inteligente ao invés de escolher ações completamente aleatórias. O servidor poderia analisar o histórico de ações do jogador nos últimos turnos e tentar prever padrões de comportamento (mais complexo), ou níveis de dificuldade configuráveis onde, dependendo do nível escolhido, o servidor usa diferentes estratégias de seleção de ações ao invés de escolha completamente aleatória, sendo mais acirrado as partidas e evitando fugas desnecessárias e aleatórias que ocorrem.

Além disso, adicionar um sistema simples de histórico de partidas seria importante para acompanhar o progresso. O servidor poderia salvar em arquivo texto as estatísticas de cada partida (vitórias, derrotas, HP médio restante, ações mais usadas) e exibir um resumo ao jogador quando ele conecta. Isso permitiria ao jogador ver sua evolução ao longo do tempo e identificar estratégias que funcionam melhor

5. Considerações Finais

O desenvolvimento deste trabalho proporcionou experiência prática valiosa na implementação de protocolos de rede personalizados e comunicação cliente-servidor usando sockets TCP/IP, especialmente na resolução de problemas relacionados à sincronização de estado distribuído e design de protocolo.

A parte mais direta de implementar foi o protocolo básico de comunicação e a estrutura de mensagens, por ser uma aplicação direta dos conceitos de sockets TCP combinado com structs em C. A implementação do suporte dual-stack IPv4/IPv6 também foi relativamente simples, utilizando o **getaddrinfo** com **AF_UNSPEC** para resolver automaticamente ambas as versões do protocolo IP.

A principal dificuldade encontrada foi na implementação da lógica de combate com todas as combinações de regras, que exigiu compreensão sobre como modelar regras assimétricas e especiais (como Laser penetrando Cloaking mas Torpedo não). Esta parte demandou várias iterações até encontrar a solução que satisfizesse todas as regras de forma elegante e correta.

A implementação da separação correta de mensagens de término (**MSG_INVENTORY**, **MSG_ESCAPE**, **MSG_GAME_OVER**) também apresentou desafios interessantes, principalmente na parte de garantir que o cliente sempre recebesse inventário completo independente do tipo de término da partida. A escolha por envio sequencial de duas mensagens (inventário seguido de tipo de término) resolveu este problema de forma limpa e conforme especificação.

Além disso, em desenvolvimento foi-se implementado um pequeno script Shell de testes automatizados para fazer várias partidas e verificar se os resultados estão corretos, cobrindo compilação, regras de combate, e conectividade IPv4/IPv6, facilitando a necessidade de demonstrando que a lógica implementada no programa está correta.

Por segurança, também foi verificado a ausência de warnings de compilação usando flags (-Wall -Wextra -pedantic) e validação manualmente do sistema através de diversas partidas para garantir seu funcionamento adequado.

6. Referências

[1] GEEKSFORGEEEKS. ****TCP Server-Client Implementation in C****. Disponível em: <https://www.geeksforgeeks.org/tcp-server-client-implementation-in-c/>

[2] GEEKSFORGEEEKS. ****Socket Programming in C/C++****. Disponível em: <https://www.geeksforgeeks.org/socket-programming-cc/>

[3] GEEKSFORGEEEKS. ****Client-Server Model****. Disponível em: <https://www.geeksforgeeks.org/system-design/client-server-model/>

[4] LINUX FOUNDATION. ****getaddrinfo(3) - Linux Manual Page****. Man Pages Project. Disponível em: <https://man7.org/linux/man-pages/man3/getaddrinfo.3.html>

[5] HALL, Brian "Beej Jorgensen". ****Beej's Guide to Network Programming: Using Internet Sockets****. 2023. Disponível em: <https://beej.us/guide/bgnet/>

[5] TANENBAUM, Andrew S.; FEAMSTER, Nick; WETHERALL, David J. ****Redes de Computadores****. 6ª ed. Pearson Education, 2021.